Architecture Diagrams

sync vs async, read and write paths, C4

Watch the 3-min video

In one line: lay boxes out left to right in layers, label every box and arrow, draw sync calls solid and async messages dashed through a visible queue, split read and write paths, and pick the right zoom level (C4).

1 A layout anyone can follow

You finish your estimates and pick up the marker. Five minutes later the board is a tangle of boxes and crossing arrows, and even you can't say which request goes where. A clear diagram is half of a system design interview.

Lay it out **from left to right, in layers**:

- **Clients** on the left: web, mobile, other services.
- The **edge**: DNS, CDN, and the load balancer or API gateway.
- The **services** in the middle.
- The **data stores** on the right: databases, caches, object storage, search.

Label every box with what it *is* ("Orders DB, Postgres", not "DB") and every arrow with what *flows* along it.

2 Sync and async look different

Use **solid arrows** for synchronous calls, where the caller waits for a response, like HTTP or gRPC. Use **dashed arrows** for asynchronous flows, like messages through a queue or a Kafka topic, where the sender doesn't wait. Draw the queue as **its own box**, so everyone sees where work can pile up. Number the steps of important flows so you can walk through them in order.

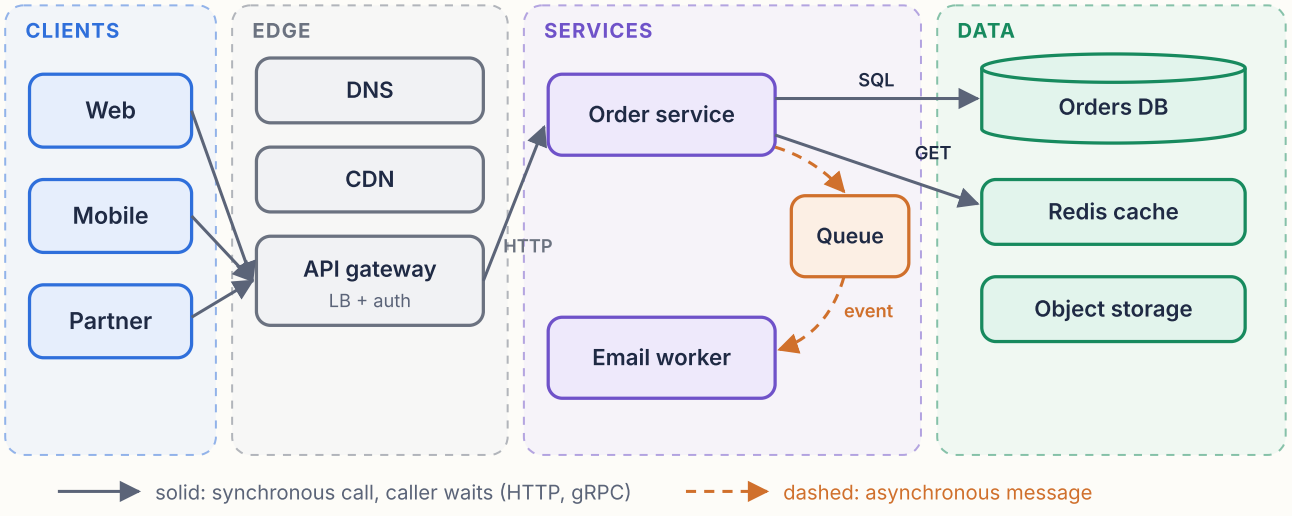


Figure 1. Four layers, left to right. The order service answers the client synchronously, and hands the email work to a queue (dashed) that a worker drains later.

3 Separate the read path from the write path

Reads and writes often use different components and scale differently. Drawing them separately, or in different colors, makes the design much easier to follow and discuss.

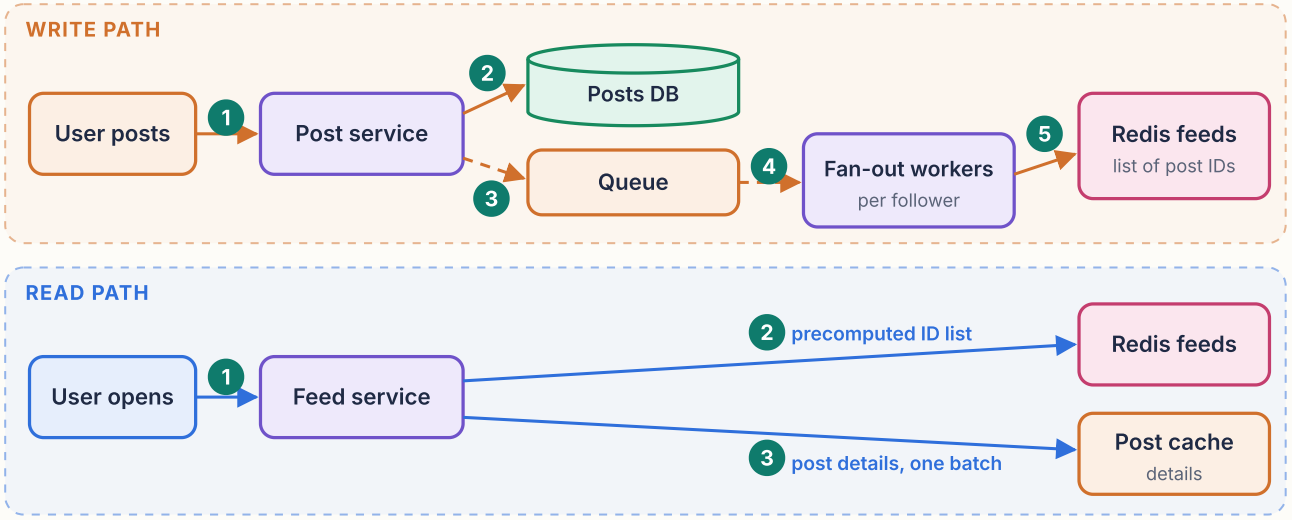


Figure 2. A news feed. Writes do the heavy work up front (fan-out on write); reads only touch Redis and a cache.

WORKED EXAMPLE: THE NEWS FEED

Write path. (1) A user posts. (2) The post service stores it in the posts database and (3) publishes an event to a queue. (4) Fan-out workers consume it and (5) push the post ID into each follower's feed list in Redis.

Read path. (1) The user opens the app. (2) The feed service reads their precomputed list from Redis, then (3) fetches post details from a cache. No heavy database queries on the read path.

4 Sequence diagrams for one important request

An architecture diagram shows *what* talks to what. A **sequence diagram** shows *when*. Participants are columns: client, gateway, service, cache, database. Time flows downward, and each call and response is a horizontal arrow. It is ideal for a checkout or a login, because you can show retries and timeouts explicitly.

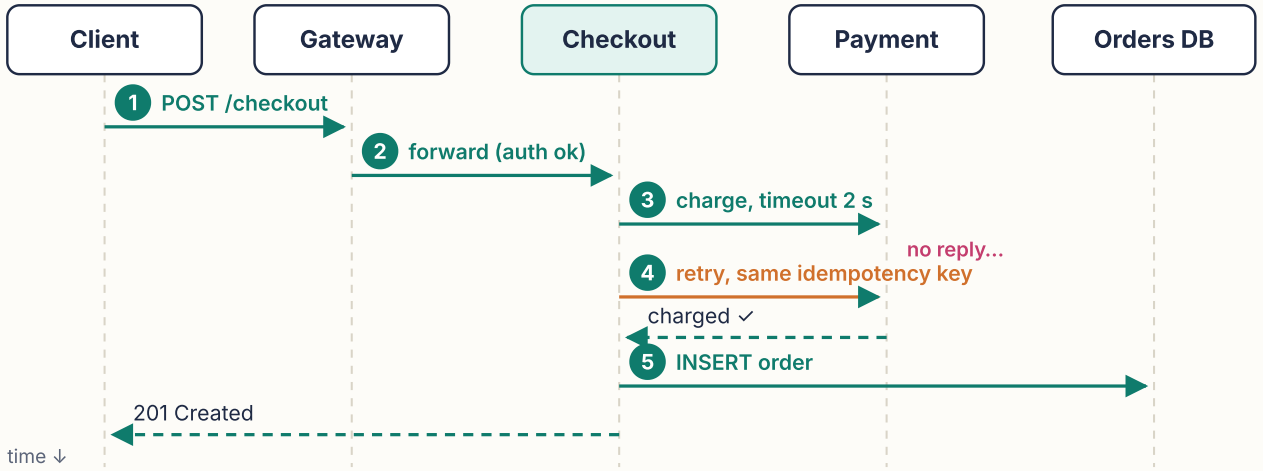


Figure 3. Checkout as a sequence. The payment call times out after 2 s and is retried with the same idempotency key, so the customer is charged once.

5 C4: four levels of zoom

The **C4 model**, from Simon Brown, gives you levels of zoom so each picture answers one question:

- **Level 1, Context:** your system as one box, with its users and the external systems it talks to.
- **Level 2, Containers:** the deployable or runnable parts, like the web app, API, database and queue.
- **Level 3, Components:** the main modules inside one container.
- **Level 4, Code:** classes, rarely drawn.

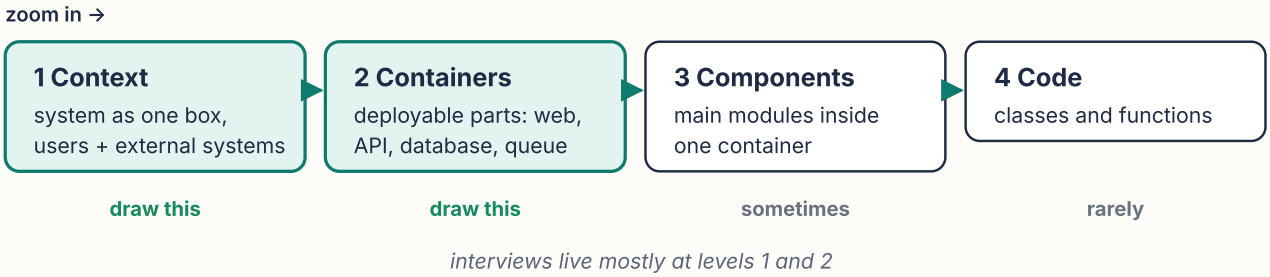


Figure 4. C4 levels. A "container" in C4 means anything that runs or stores data on its own, not specifically a Docker container.

COMMON MISTAKES

- Unlabelled arrows: nobody knows whether it's a call, a message or a replication stream.
- Drawing a queue as a line instead of a box, so its backlog and consumers are invisible.
- One giant diagram mixing reads, writes, background jobs and admin flows.
- Jumping to class-level detail (C4 level 4) before the containers are agreed.
- Arrows that cross the whole board; re-lay the boxes so flows move left to right.

6 Key terms

Synchronous: caller waits for the response

Asynchronous: sender hands off a message and moves on

Read path / write path: the components a read or a write touches

Fan-out on write: copy a new item into each follower's feed at write time

Sequence diagram: participants as columns, time flowing down

C4 model: context, containers, components, code

7 Interview questions

Q1. How do you show sync vs async flows on a diagram? BASIC

Solid arrows for synchronous request-response calls (HTTP, gRPC). Dashed arrows through an explicit queue or topic box for asynchronous messages. Number the steps.

Q2. What is the C4 model? BASIC

Simon Brown's four levels of zoom: context (system + users + external systems), containers (deployable parts), components (modules inside one container), code (classes). Interviews use levels 1 and 2.

Q3. How do you lay out a system diagram? BASIC

Left to right in layers: clients, edge (DNS, CDN, load balancer or gateway), services, data stores. Label every box with what it is and every arrow with what flows.

Q4. Draw the write and read paths for a news feed. SCENARIO

Write: post service stores the post, publishes to a queue, fan-out workers push the post ID into each follower's Redis list. Read: feed service reads the precomputed list, fetches details from a cache.

Q5. When would you add a sequence diagram? SCENARIO

For one critical request with several hops, like checkout or login, where order, timeouts and retries matter. Example: payment times out at 2 s, is retried with the same idempotency key.

Q6. Why draw the queue as its own box? SCENARIO

It is a real component: it can fill up, needs consumers, and has its own scaling and failure modes. As a box you can label its throughput, say 5,000 messages/s, and its consumers.

Q7. Why separate read and write paths if they share a database? TRICKY

They scale differently. A 100 : 1 read-heavy system adds replicas and caches to the read path while the write path stays on one primary. Separate drawings make that obvious.

Q8. Is a C4 "container" a Docker container? TRICKY

No. It is any separately running or data-storing unit: a web app, a mobile app, a database, a queue. It may or may not be packaged in Docker.

Next: SD-D-013 · Identifying Bottlenecks backendroadmap.com